

# Navegación de costo mínimo sobre terreno real mediante Aprendizaje por Refuerzo Profundo

Integrante 1, Integrante 2, Integrante 3,  
Integrante 4, Integrante 5  
Maestría en Inteligencia Artificial (MIA-204)  
Aprendizaje por Refuerzo  
Arequipa, Perú

**Resumen**—Un agente aprende a cruzar un terreno real (las faldas del volcán Misti, en Arequipa) gastando el menor esfuerzo por pendiente. En el trabajo parcial usamos Q-learning tabular con una meta fija. Aquí subimos la dificultad y aplicamos Aprendizaje por Refuerzo Profundo. El mapa va a resolución completa ( $108 \times 108$  celdas de 30 m), el agente solo ve un parche  $9 \times 9$  de pendiente a su alrededor más un vector hacia la meta, y el punto de partida cambia en cada episodio. La función de valor la aproxima una red neuronal (Deep Q-Network, DQN). Comparamos dos familias de métodos, los basados en valor (DQN y Double DQN) y los basados en política (REINFORCE), y medimos contra el óptimo exacto de Dijkstra. El agente llega a la meta desde puntos de partida que nunca vio en entrenamiento, con una ruta cercana al óptimo, y sigue funcionando cuando el mundo se vuelve estocástico. Reportamos también dos problemas de diseño y cómo los resolvimos: el *reward shaping* potencial de libro creaba una trampa que impedía aprender, y lo cambiamos por *shaping* de progreso; y con metas totalmente aleatorias los valores  $Q$  divergen, lo que explica por qué elegimos meta fija.

**Index Terms**—Aprendizaje por Refuerzo Profundo, Deep Q-Network, REINFORCE, navegación, reward shaping, modelo digital de elevación.

## I. INTRODUCCIÓN

Cruzar un terreno gastando el menor esfuerzo es un problema viejo. Si conoces el mapa completo, lo resuelves exacto con Dijkstra. Pero un agente autónomo casi nunca tiene el mapa entero: ve su alrededor y decide con información parcial. El Aprendizaje por Refuerzo (Reinforcement Learning, RL) sirve justo para ese caso: aprende una política por prueba y error.

En el trabajo parcial modelamos el terreno como un Proceso de Decisión de Markov (Markov Decision Process, MDP) y lo resolvimos con Q-learning tabular. La malla era reducida ( $54 \times 54$ ), y el inicio y la meta eran fijos. Ese enfoque guarda una tabla de valores y no sirve para situaciones nuevas: memoriza, no generaliza.

En este trabajo subimos la dificultad y usamos Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning, DRL). Aportamos tres cosas:

- Un entorno más difícil: resolución completa ( $108 \times 108$ ), observación local (un parche  $9 \times 9$  más el vector a la meta) e inicio aleatorio. Esto obliga al agente a generalizar y justifica usar una red neuronal en lugar de una tabla.

- Una comparación de dos familias de RL en el mismo terreno: basada en valor (DQN, Double DQN) y basada en política (REINFORCE), más la línea base exacta de Dijkstra.
- Dos problemas de diseño que encontramos y resolvimos, con datos que lo respaldan: la trampa del *shaping* potencial y la divergencia de  $Q$  con metas aleatorias.

## II. MARCO TEÓRICO

### II-A. MDP y Q-learning

Un MDP se define por estados  $s$ , acciones  $a$ , una transición  $P(s'|s, a)$  y una recompensa  $r(s, a)$ . El objetivo es una política  $\pi$  que maximice el retorno descontado  $\sum_t \gamma^t r_t$ . Q-learning aprende la función de valor de acción  $Q(s, a)$  con la actualización por diferencia temporal (Temporal Difference, TD):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (1)$$

La política greedy es  $\pi(s) = \arg \max_a Q(s, a)$ .

### II-B. Deep Q-Network (DQN)

Cuando hay muchos estados, la tabla  $Q$  no cabe. DQN [1] la reemplaza por una red neuronal  $Q(s, a; \theta)$  y minimiza la pérdida

$$\mathcal{L}(\theta) = (r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2, \quad (2)$$

por descenso de gradiente. Entrenar así, sin más, es inestable. DQN lo estabiliza con dos mecanismos de la clase 5.1:

- **Reproducción de Experiencias** (experience replay): guarda las transiciones y entrena con mini-lotes al azar, para romper la correlación entre pasos seguidos.
- **Objetivos-Q Fijos** (red objetivo  $\theta^-$ ): una copia congelada de la red calcula el objetivo, y se sincroniza cada cierto número de pasos. Así el agente no persigue un blanco que se mueve con cada update.

### II-C. Double DQN

El operador  $\max$  del objetivo tiende a sobreestimar los valores, porque siempre elige el ruido positivo. Double DQN [2] separa quién elige la acción de quién la evalúa:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-), \quad (3)$$

lo que reduce ese sesgo.

## II-D. REINFORCE (gradiente de política)

REINFORCE [5] no aprende valores  $Q$ . Aprende directamente una política estocástica  $\pi(a|s; \theta)$  y ajusta sus parámetros hacia donde crece el retorno:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi(a_t|s_t; \theta) G_t, \quad (4)$$

donde  $G_t$  es el retorno descontado desde el paso  $t$ . Restar una línea base a  $G_t$  baja la varianza, que es su punto débil.

## II-E. Reward shaping

Cuando la recompensa es dispersa, un bono denso ayuda a aprender. El *shaping* potencial [4] promete no cambiar la política óptima:  $F = \gamma \Phi(s') - \Phi(s)$ . En nuestro problema, con  $\gamma < 1$  y rutas largas, esa forma trae un efecto que no queríamos (Sec. V).

## III. METODOLOGÍA

### III-A. El terreno y el MDP

Usamos un recorte de unos  $3 \times 3$  km del modelo digital de elevación (Digital Elevation Model, DEM) Copernicus GLO-30, bajado de AWS Open Data. La Fig. 1 muestra la elevación y la pendiente. El MDP es:

- **Estados:** el agente está en una celda de la malla  $108 \times 108$ .
- **Acciones:** arriba, abajo, izquierda, derecha.
- **Transición:** determinística, u opcionalmente estocástica (el resbalón de la Sec. IV-D).
- **Recompensa:** un costo por paso que crece con la pendiente, un castigo si choca contra un obstáculo, y un premio al llegar a la meta.

### III-B. Observación local

El agente no ve el mapa completo. Ve un parche de  $9 \times 9$  celdas de pendiente centrado en él (81 valores) más el vector  $(\Delta \text{fila}, \Delta \text{columna})$  hacia la meta, en total 83 números (Fig. 2). Ese tamaño no depende del mapa. Y como terrenos parecidos dan observaciones parecidas, la red puede generalizar. Una tabla no puede: necesitaría una entrada por cada par (posición, meta), del orden de  $10^8$  valores.

### III-C. Meta fija con inicio aleatorio

La meta es siempre la misma celda; el inicio cambia en cada episodio. Con una sola meta, la función de valor es un campo suave y el entrenamiento se mantiene estable. El inicio aleatorio hace el problema no trivial: el agente aprende a llegar desde cualquier celda, no una ruta memorizada.

### III-D. Reward shaping de progreso

Con la meta lejos en un mapa grande, el agente casi nunca la encuentra por azar y el premio nunca aparece. Le damos un bono por acercarse. Primero probamos el *shaping* potencial  $\Phi(s) = -k \cdot d(s, \text{meta})$ , y falló (Sec. V). Lo cambiamos por **shaping de progreso**,  $r += k [d(s, \text{meta}) - d(s', \text{meta})]$ , que suma cero en cualquier ciclo y no crea la trampa del potencial.

Cuadro I  
ABLACIONES: TASA DE ÉXITO EN EL TEST

| Configuración                   | Éxito |
|---------------------------------|-------|
| DQN completo                    | 100 % |
| sin <i>replay</i>               | 85 %  |
| sin red objetivo ( $\theta^-$ ) | 0 %   |
| Double DQN                      | 100 % |

## III-E. Arquitectura e hiperparámetros

La red (tanto la de  $Q$  como la de política) es un perceptrón de  $83 \rightarrow 128 \rightarrow 128 \rightarrow 4$ , unos 27 mil parámetros. Usamos  $\gamma = 0,99$ , optimizador Adam, pérdida de Huber y recorte de gradiente en DQN. La exploración es  $\epsilon$ -greedy decreciente. Un *curriculum* hace que los inicios arranquen cerca de la meta y se alejen con el entrenamiento. Fijamos la semilla en todos los experimentos para que sean reproducibles.

## IV. EXPERIMENTOS Y RESULTADOS

### IV-A. Configuración

El test mide dos cosas de la política greedy: cuántas veces llega a la meta y en cuántos pasos, sobre 20 inicios fijos sorteados con una semilla distinta a la del entrenamiento. Es decir, inicios que el agente nunca vio. Además comparamos el costo de la ruta del agente contra el óptimo exacto de Dijkstra.

### IV-B. DQN con meta fija

La Fig. 3 muestra la curva de aprendizaje. El agente llega al **100 % de éxito** (20 de 20 inicios) y se queda estable. La Fig. 4 pone sus rutas al lado de las de Dijkstra: la brecha media de costo es de solo **7,1 %**, aunque el agente nunca vio el mapa completo, solo parches locales.

### IV-C. Ablaciones

Repetimos el estudio de ablación de Liu & Zou [3] en nuestro terreno: apagamos cada mecanismo de DQN y medimos qué pasa (Tabla I, Fig. 5). El resultado es claro. Sin la red objetivo ( $\theta^-$ ), el agente no aprende nada (0 %). Sin la Reproducción de Experiencias, aprende pero peor (85 %). Los dos componentes hacen falta. Double DQN queda a la par del DQN completo.

### IV-D. Transición estocástica

Volvemos el mundo estocástico con un resbalón: con una probabilidad que crece con la pendiente, la acción del agente se cambia por una al azar. La Fig. 6 compara terreno determinista y estocástico. En ambos casos el agente llega con alta tasa de éxito (95–100 %) y en un número de pasos parecido (unos 103–113). La política aguanta el azar, algo que Dijkstra no modela porque supone que el mundo es perfecto.

#### IV-E. Valor vs política: DQN y REINFORCE

La Fig. 7 compara DQN (basado en valor) con REINFORCE (basado en política) en el mismo terreno. Los dos llegan alto: DQN 100 % y REINFORCE 95 %. La diferencia está en cómo. DQN converge suave y estable, gracias a la Reproducción de Experiencias y la red objetivo. REINFORCE es más simple pero tiene mucha más varianza: la recompensa por episodio salta de un valor a otro. Es el comportamiento típico de cada familia.

#### IV-F. Divergencia con meta aleatoria

La Fig. 8 muestra qué pasa si la meta también es aleatoria (problema *goal-conditioned*). La magnitud media de los valores  $Q$  se mantiene baja hasta cerca del episodio 1000 y luego **explota**: pasa de  $\sim 1$  a más de 2400. Mientras tanto la tasa de éxito nunca sube de 10 %. El agente no aprende una política que se sostenga, y los valores divergen por la sobreestimación del operador máx. Esta es la razón por la que elegimos meta fija.

### V. DISCUSIÓN

#### V-A. La trampa del shaping potencial

El *shaping* potencial con  $\gamma < 1$  deja un residuo positivo por estar lejos de la meta. Sobre un ciclo entre dos celdas, la recompensa de *shaping* telescopa a  $k(1 - \gamma)(d_X + d_Y) > 0$ . A distancias grandes eso significa que oscilar lejos de la meta (ir y volver, *ping-pong*) da recompensa positiva, así que la política greedy se queda atrapada yendo y viniendo entre dos celdas. El *shaping* de progreso quita ese residuo y el problema desaparece. Es un caso donde una técnica correcta en teoría, la invariancia de la política óptima, falla en la práctica por cómo interactúa con el descuento.

#### V-B. Divergencia y la elección de meta fija

Con metas aleatorias, la sobreestimación del máx se realimenta con el *bootstrapping* y los valores  $Q$  se disparan. Double DQN retrasa el efecto, pero en nuestro caso no lo evita. La meta fija da un campo de valor único y estable. No es una limitación accidental, es una decisión de alcance con motivo.

#### V-C. Limitación: política single-goal

Nuestra política generaliza sobre el inicio, pero está entrenada para una meta fija. Si se cambia la meta, hay que reentrenar. Llevarla a metas cualesquiera pide técnicas más robustas contra la divergencia, y lo dejamos como trabajo futuro.

### VI. CONCLUSIONES Y TRABAJO FUTURO

Construimos un agente de Aprendizaje por Refuerzo Profundo que navega terreno real con observación local e inicio aleatorio, y aproxima la función de valor con una red neuronal. El agente llega a la meta desde inicios que nunca vio, con una ruta cercana al óptimo de Dijkstra, y aguanta las transiciones estocásticas. Comparamos las familias de valor y de política, y reportamos dos problemas de diseño con datos que los

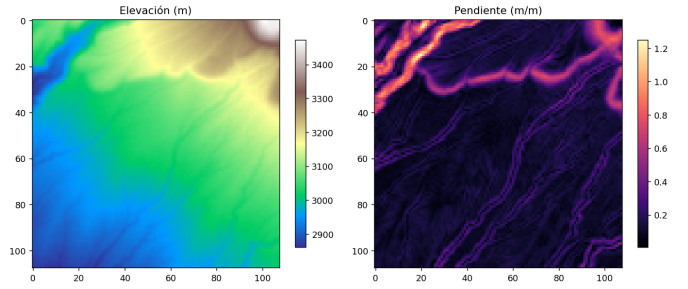


Figura 1. Terreno real (faldas del Misti): elevación (izq.) y pendiente (der.), malla  $108 \times 108$  de 30 m.

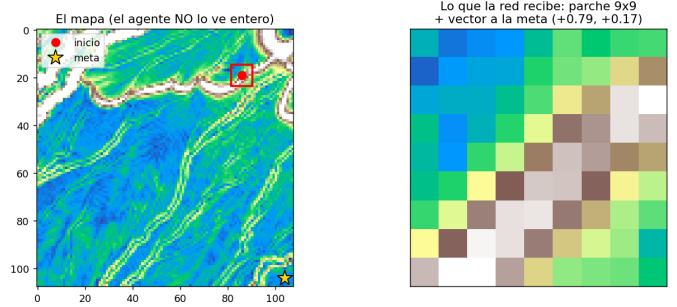


Figura 2. Observación local: el mapa completo con la ventana del agente (izq.) y el parche  $9 \times 9$  que recibe la red (der.), más el vector a la meta.

respaldan. Para el futuro proponemos tres líneas: políticas condicionadas a la meta (*goal-conditioned*) con Hindsight Experience Replay o replay priorizado; mapas más grandes, como el volcán completo; y métodos actor-critic.

### REFERENCIAS

- [1] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [2] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *AAAI*, 2016.
- [3] R. Liu and J. Zou, “The effects of memory replay in reinforcement learning,” in *Allerton Conf.*, IEEE, 2018.
- [4] A. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: theory and application to reward shaping,” in *ICML*, 1999.
- [5] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

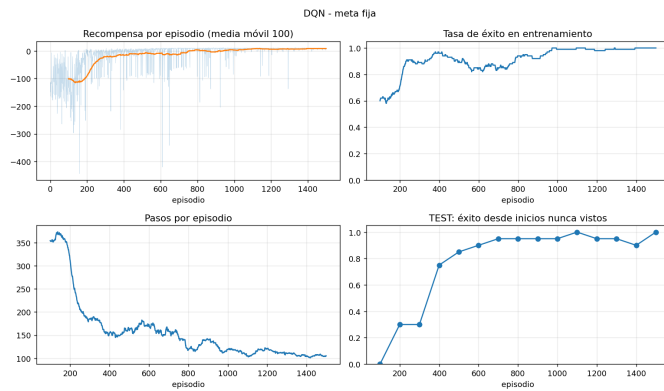


Figura 3. Curva de aprendizaje de DQN con meta fija: recompensa, tasa de éxito de entrenamiento, pasos y test sobre inicios no vistos.

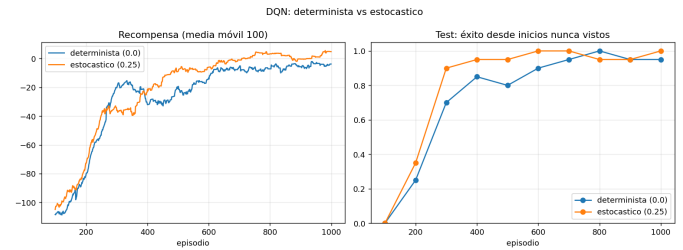


Figura 6. DQN en terreno determinista vs estocástico (resbalón 0,25).

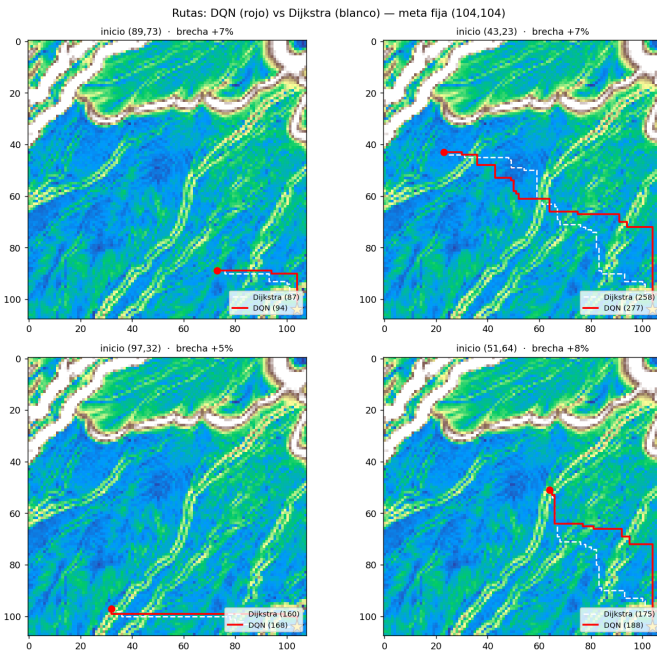


Figura 4. Rutas aprendidas por DQN (rojo) frente al óptimo exacto de Dijkstra (blanco), para cuatro pares inicio–meta de test.

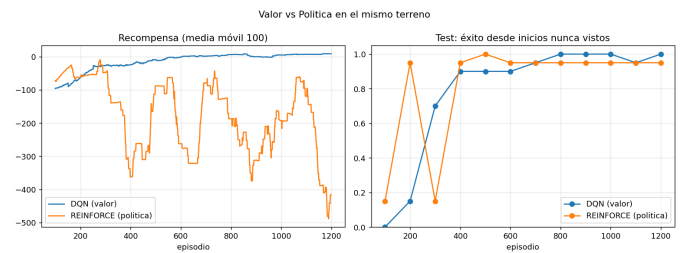


Figura 7. Valor vs política: DQN y REINFORCE en el mismo terreno.

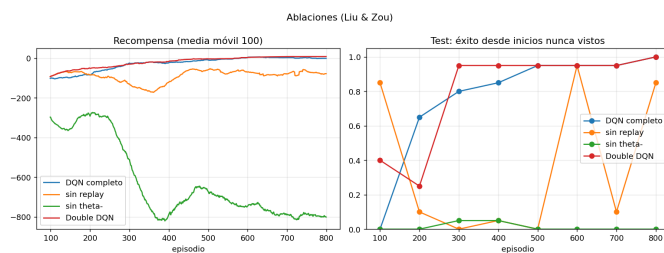


Figura 5. Ablaciones: DQN completo, sin *replay*, sin red objetivo ( $\theta^-$ ) y Double DQN.

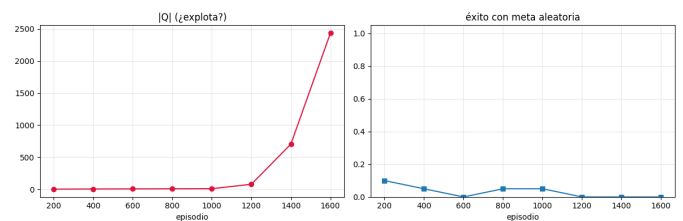


Figura 8. Con meta aleatoria, la magnitud media de  $Q$  (izq.) crece sin control (divergencia) y el éxito (der.) no se sostiene.